

be used to index in-memory data, with no change to the structure. However, some optimizations are possible.

First, since memory is costlier than disk space, internal data structures in main memory databases have to be designed to reduce space requirements. Techniques that we saw in Section 14.4.1 to improve B⁺-tree storage utilization can be used to reduce memory usage for in-memory B⁺-trees.

Data structures that require traversal of multiple pointers are acceptable for in-memory data, unlike in the case of disk-based data, where the cost of the I/Os to traverse multiple pages would be excessively high. Thus, tree structures in main memory databases can be relatively deep, unlike B⁺-trees.

The speed difference between cache memory and main memory, and the fact that data are transferred between main memory and cache in units of a *cache-line* (typically about 64 bytes), results in a situation where the relationship between cache and main memory is not dissimilar to the relationship between main memory and disk (although with smaller speed differences). When reading a memory location, if it is present in cache the CPU can complete the read in 1 or 2 nanoseconds, whereas a cache miss results in about 50 to 100 nanoseconds of delay to read data from main memory.

B⁺-trees with small nodes that fit in a cache line have been found to provide very good performance with in-memory data. Such B⁺-trees allow index operations to be completed with far fewer cache misses than tall, skinny tree structures such as binary trees, since each node traversal is likely to result in a cache miss. Compared to B⁺-trees with nodes that match cache lines, trees with large nodes also tend to have more cache misses since locating data within a node requires either a full scan of the node content, spanning multiple cache lines, or a binary search, which also results in multiple cache misses.

For databases where data do not fit entirely in memory, but frequently used data are often memory resident, the following idea is used to create B⁺-tree structures that offer good performance on disk as well as in-memory. Large nodes are used to optimize disk-based access, but instead of treating data in a node as single large array of keys and pointers, the data within a node are structured as a tree, with smaller nodes that match the size of a cache line. Instead of scanning data linearly or using binary search within a node, the tree-structure within the large B⁺-tree node is used to access the data with a minimal number of cache misses.

14.5 Hash Indices

Hashing is a widely used technique for building indices in main memory; such indices may be transiently created to process a join operation (as we will see in Section 15.5.5) or may be a permanent structure in a main memory database. Hashing has also been used as a way of organizing records in a file, although hash file organizations are not very widely used. We initially consider only in-memory hash indices, and we consider disk-based hashing later in this section.

In our description of hashing, we shall use the term **bucket** to denote a unit of storage that can store one or more records. For in-memory hash indices, a bucket could be a linked list of index entries or records. For disk-based indices, a bucket would be a linked list of disk blocks. In a **hash file organization**, instead of record pointers, buckets store the actual records; such structures only make sense with disk-resident data. The rest of our description does not depend on whether the buckets store record pointers or actual records.

Formally, let K denote the set of all search-key values, and let B denote the set of all bucket addresses. A **hash function** h is a function from K to B . Let h denote a hash function. With in-memory hash indices, the set of buckets is simply an array of pointers, with the i th bucket at offset i . Each pointer stores the head of a linked list containing the entries in that bucket.

To insert a record with search key K_i , we compute $h(K_i)$, which gives the address of the bucket for that record. We add the index entry for the record to the list at offset i . Note that there are other variants of hash indices that handle the case of multiple records in a bucket differently; the form described here is the most widely used variant and is called **overflow chaining**.

Hash indexing using overflow chaining is also called **closed addressing** (or, less commonly, **closed hashing**). An alternative hashing scheme called open addressing is used in some applications, but is not suitable for most database indexing applications since open addressing does not support deletes efficiently. We do not consider it further.

Hash indices efficiently support equality queries on search keys. To perform a lookup on a search-key value K_i , we simply compute $h(K_i)$, then search the bucket with that address. Suppose that two search keys, K_5 and K_7 , have the same hash value; that is, $h(K_5) = h(K_7)$. If we perform a lookup on K_5 , the bucket $h(K_5)$ contains records with search-key values K_5 and records with search-key values K_7 . Thus, we have to check the search-key value of every record in the bucket to verify that the record is one that we want.

Unlike B⁺-tree indices, hash indices do not support range queries; for example, a query that wishes to retrieve all search key values v such that $l \leq v \leq u$ cannot be efficiently answered using a hash index.

Deletion is equally straightforward. If the search-key value of the record to be deleted is K_i , we compute $h(K_i)$, then search the corresponding bucket for that record and delete the record from the bucket. With a linked list representation, deletion from the linked list is straightforward.

In a disk-based hash index, when we insert a record, we locate the bucket by using hashing on the search key, as described earlier. Assume for now that there is space in the bucket to store the record. Then, the record is stored in that bucket. If the bucket does not have enough space, a **bucket overflow** is said to occur. We handle bucket overflow by using **overflow buckets**. If a record must be inserted into a bucket b , and b is already full, the system provides an overflow bucket for b and inserts the record into the overflow bucket. If the overflow bucket is also full, the system provides another overflow bucket, and so on. All the overflow buckets of a given bucket are chained together in a linked

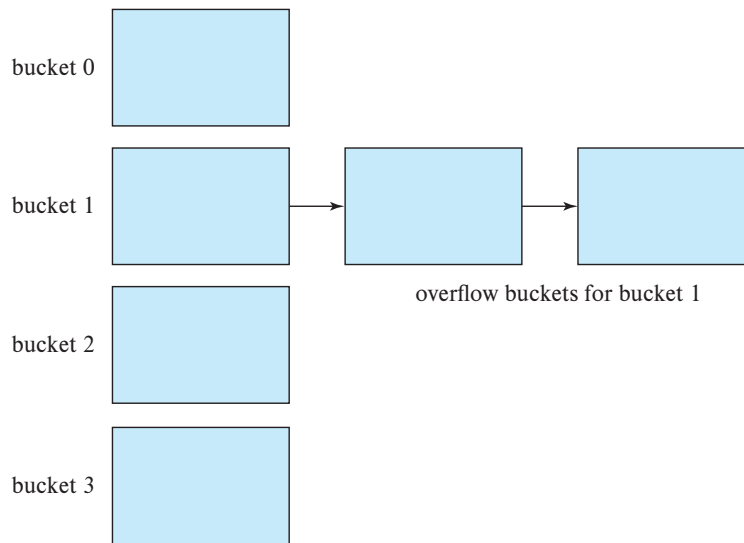


Figure 14.25 Overflow chaining in a disk-based hash structure.

list, as in Figure 14.25. With overflow chaining, given search key k , the lookup algorithm must then search not only bucket $h(k)$, but also the overflow buckets linked from bucket $h(k)$.

Bucket overflow can occur if there are insufficient buckets for the given number of records. If the number of records that are indexed is known ahead of time, the required number of buckets can be allocated; we will shortly see how to deal with situations where the number of records becomes significantly more than what was initially anticipated. Bucket overflow can also occur if some buckets are assigned more records than are others, resulting in one bucket overflowing even when other buckets still have a lot of free space.

Such [skew](#) in the distribution of records can occur if multiple records may have the same search key. But even if there is only one record per search key, skew may occur if the chosen hash function results in nonuniform distribution of search keys. This chance of this problem can be minimized by choosing hash functions carefully, to ensure the distribution of keys across buckets is uniform and random. Nevertheless, some skew may occur.

So that the probability of bucket overflow is reduced, the number of buckets is chosen to be $(n_r/f_r) * (1 + d)$, where n_r denotes the number of records, f_r denotes the number of records per bucket, d is a fudge factor, typically around 0.2. With a fudge factor of 0.2, about 20 percent of the space in the buckets will be empty. But the benefit is that the probability of overflow is reduced.

Despite allocation of a few more buckets than required, bucket overflow can still occur, especially if the number of records increases beyond what was initially expected.

full. However, querying may be slower, since multiple paths have to be searched. Spatial joins are simpler with quadrees than with R-trees, since all quadrees on a region are partitioned in the same manner. However, because of their better storage efficiency and their similarity to B-trees, R-trees and their variants have proved popular in database systems that support spatial data.

24.5 Hash Indices

We described the concepts of hashing and hash indices in Section 14.5. We provide further details in this section.

24.5.1 Static Hashing

As in Section 14.5, let K denote the set of all search-key values, and let B denote the set of all bucket addresses. A hash function h is a function from K to B . Let h denote a hash function. Recall that in a hash index, buckets contain index entries, with pointers to records, while in a hash file organization, actual records are stored in the buckets. All the other details remain the same, so we do not explicitly differentiate between these two versions henceforth. We use the term *hash index* to denote hash file organizations as well as secondary hash indices.

Figure 24.6 shows a secondary hash index on the *instructor* file, for the search key *ID*. The hash function in the figure computes the sum of the digits of the *ID* modulo 8. The hash index has eight buckets, each of size 2 (realistic indices would have much larger bucket sizes). One of the buckets has three keys mapped to it, so it has an overflow bucket. In this example, *ID* is a primary key for *instructor*, so each search key has only one associated pointer. In general, multiple pointers can be associated with each key.

Hash indices can efficiently answer *point queries*, which retrieve records with a specified value for a search key. However, they cannot efficiently answer *range queries*, which retrieve all records whose search key value lies in a range (lb, ub) . The difficulty arises because a good hash function assigns values randomly to buckets. Thus, there is no simple notion of “next bucket in sorted order.” The reason we cannot chain buckets together in sorted order on A_i is that each bucket is assigned many search-key values. Since values are scattered randomly by the hash function, the values in the specified range are likely to be scattered across many or all of the buckets. Therefore, we have to read all the buckets to find the required search keys.

Recall that deletion is done as follows: If the search-key value of the record to be deleted is K_i , we compute $h(K_i)$, then search the corresponding bucket for that record, and delete the record from the bucket. Deletion of a record is efficient if there are not many records with a given key value. However, in the case of a hash index on a key with many duplicates, a large number of entries with the same key value may have to be scanned to find the entry for the record that is to be deleted. The complexity can in the worst case be linear in the number of records.

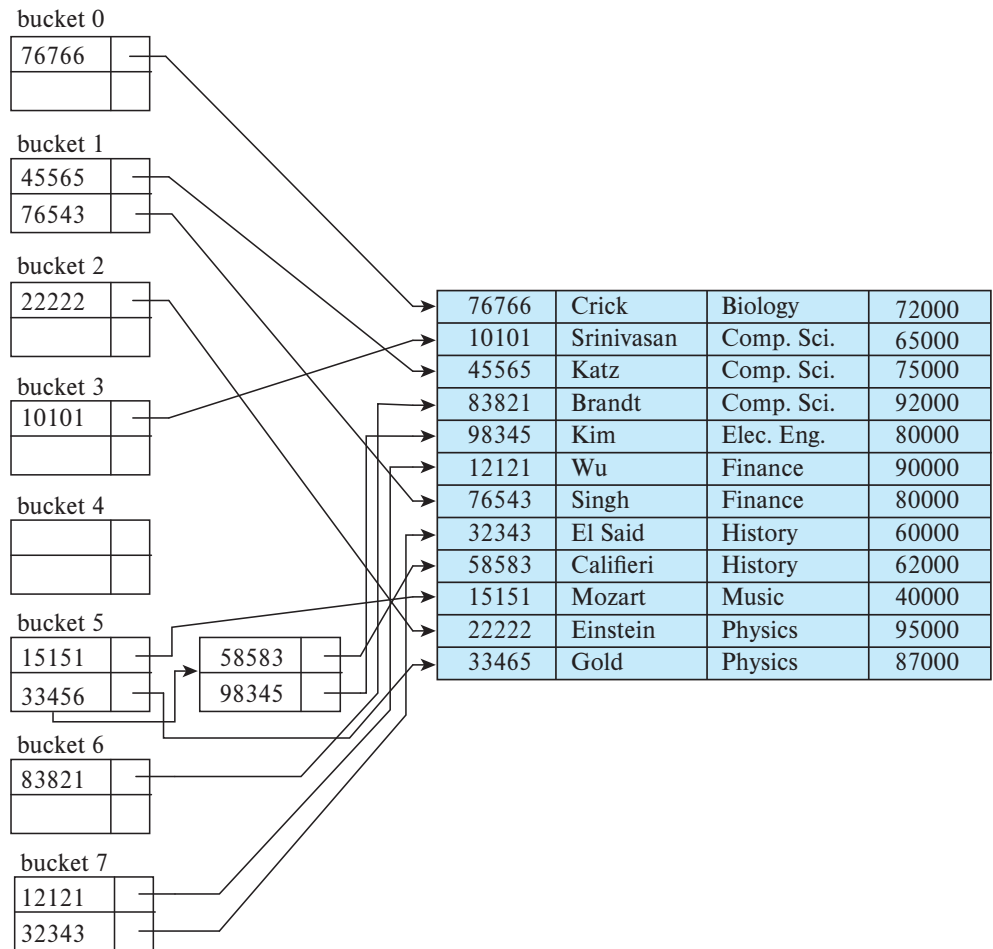


Figure 24.6 Hash index on search key *ID* of *instructor* file.

Recall also that with static hashing, the set of buckets is fixed at the time the index is created. If the relation grows far beyond the expected size, hash indices would be quite inefficient due to long overflow chains. We could rebuild the hash index using a larger number of buckets. Such rebuilding can be triggered when the number of records exceeds the estimated number by some margin, and the index is rebuilt with a number of buckets that is a multiple of the original number of buckets (say by a factor of 1.5 to 2). Such rebuilding is in fact done in many systems with in-memory hash indices.

However, doing so can cause significant disruption to normal processing with large relations, since a large number of records have to be reindexed; the disruption is particularly marked with disk-resident data. In this section we discuss dynamic hashing techniques that allow hash indices to grow gradually, without causing disruption.

24.5.1.1 Hash Functions

The worst possible hash function maps all search-key values to the same bucket. Such a function is undesirable because all the records have to be kept in the same bucket. A lookup has to examine every such record to find the one desired. An ideal hash function distributes the stored keys uniformly across all the buckets, so that every bucket has the same number of records.

Since we do not know at design time precisely which search-key values will be stored in the file, we want to choose a hash function that assigns search-key values to buckets in such a way that the distribution has these qualities:

- The distribution is *uniform*. That is, the hash function assigns each bucket the same number of search-key values from the set of *all* possible search-key values.
- The distribution is *random*. That is, in the average case, each bucket will have nearly the same number of values assigned to it, regardless of the actual distribution of search-key values. More precisely, the hash value will not be correlated to any externally visible ordering on the search-key values, such as alphabetic ordering or ordering by the length of the search keys; the hash function will appear to be random.

As an illustration of these principles, let us choose a hash function for the *instructor* file using the search key *dept_name*. The hash function that we choose must have the desirable properties not only on the example *instructor* file that we have been using, but also on an *instructor* file of realistic size for a large university with many departments.

Assume that we decide to have 26 buckets, and we define a hash function that maps names beginning with the *i*th letter of the alphabet to the *i*th bucket. This hash function has the virtue of simplicity, but it fails to provide a uniform distribution, since we expect more names to begin with such letters as B and R than Q and X, for example.

Now suppose that we want a hash function on the search key *salary*. Suppose that the minimum salary is \$30,000 and the maximum salary is \$130,000, and we use a hash function that divides the values into 10 ranges, \$30,000–\$40,000, \$40,001–\$50,000, and so on. The distribution of search-key values is uniform (since each bucket has the same number of different *salary* values) but is not random. Records with salaries between \$60,001 and \$70,000 are far more common than are records with salaries between \$30,001 and \$40,000. As a result, the distribution of records is not uniform—some buckets receive more records than others do. If the function has a random distribution, even if there are such correlations in the search keys, the randomness of the distribution will make it very likely that all buckets will have roughly the same number of records, as long as each search key occurs in only a small fraction of the records. (If a single search key occurs in a large fraction of the records, the bucket containing it is likely to have more records than other buckets, regardless of the hash function used.)

bucket 0			

bucket 1			
15151	Mozart	Music	40000

bucket 2			
32343	El Said	History	80000
58583	Califieri	History	60000

bucket 3			
22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

bucket 4			
12121	Wu	Finance	90000
76543	Singh	Finance	80000

bucket 5			
76766	Crick	Biology	72000

bucket 6			
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

bucket 7			

Figure 24.7 Hash organization of *instructor* file, with *dept_name* as the key.

Typical hash functions perform computation on the internal binary machine representation of characters in the search key. A simple hash function of this type first computes the sum of the binary representations of the characters of a key, then returns the sum modulo the number of buckets.

Figure 24.7 shows the application of such a scheme, with eight buckets, to the *instructor* file, under the assumption that the *i*th letter in the alphabet is represented by the integer *i*.

The following hash function is a better alternative for hashing strings. Let *s* be a string of length *n*, and let *s*[*i*] denote the *i*th byte of the string. The hash function is defined as:

$$s[0] * 31^{(n-1)} + s[1] * 31^{(n-2)} + \dots + s[n-1]$$

The function can be implemented efficiently by setting the hash result initially to 0 and iterating from the first to the last character of the string, at each step multiplying the hash value by 31 and then adding the next character (treated as an integer). The above expression would appear to result in a very large number, but it is actually computed with fixed-size positive integers; the result of each multiplication and addition is thus automatically computed modulo the largest possible integer value plus 1. The result of the above function modulo the number of buckets can then be used for indexing.

Hash functions require careful design. A bad hash function may result in lookup taking time proportional to the number of search keys in the file. A well-designed function gives an average-case lookup time that is a (small) constant, independent of the number of search keys in the file.

24.5.1.2 Handling of Bucket Overflows

So far, we have assumed that, when a record is inserted, the bucket to which it is mapped has space to store the record. If the bucket does not have enough space, a **bucket overflow** is said to occur. Bucket overflow can occur for several reasons, as we outlined in Section 14.5.

- **Insufficient buckets.** The number of buckets, which we denote n_B , must be chosen such that $n_B > n_r/f_r$, where n_r denotes the total number of records that will be stored and f_r denotes the number of records that will fit in a bucket. This designation assumes that the total number of records is known when the hash function is chosen.
- **Skew.** Some buckets are assigned more records than are others, so a bucket may overflow even when other buckets still have space. This situation is called **bucket skew**. Skew can occur for two reasons:
 1. Multiple records may have the same search key.
 2. The chosen hash function may result in nonuniform distribution of search keys.

So that the probability of bucket overflow is reduced, the number of buckets is chosen to be $(n_r/f_r) * (1 + d)$, where d is a fudge factor, typically around 0.2. Some space is wasted: About 20 percent of the space in the buckets will be empty. But the benefit is that the probability of overflow is reduced.

Despite allocation of a few more buckets than required, bucket overflow can still occur. As we saw in Section 14.5, we handle bucket overflow by using overflow buckets. We must also change the lookup algorithm slightly to handle overflow chaining, to look at the overflow buckets in addition to the main bucket.

The form of hash structure that we have just described is called **closed addressing** (or, less commonly, **closed hashing**). Under an alternative approach called **open addressing** (or, less commonly, **open hashing**), the set of buckets is fixed, and there are no overflow chains. Instead, if a bucket is full, the system inserts records in some other bucket in the initial set of buckets B . One policy is to use the next bucket (in cyclic order) that has space; this policy is called *linear probing*. Other policies, such as computing further hash functions, are also used. Open addressing has been used to construct symbol tables for compilers and assemblers, but closed addressing is preferable for database systems. The reason is that deletion under open addressing is troublesome. Usually, compilers and assemblers perform only lookup and insertion operations on

their symbol tables. However, in a database system, it is important to be able to handle deletion as well as insertion. Thus, open addressing is of only minor importance in database implementation.

An important drawback to the form of hashing that we have described is that we must choose the hash function when we implement the system, and it cannot be changed easily thereafter if the file being indexed grows or shrinks. Since the function h maps search-key values to a fixed set B of bucket addresses, we waste space if B is made large to handle future growth of the file. If B is too small, the buckets contain records of many different search-key values, and bucket overflows can occur. As the file grows, performance suffers. We study in Section 24.5.2 how the number of buckets and the hash function can be changed dynamically.

24.5.2 Dynamic Hashing

As we have seen, the need to fix the set B of bucket addresses presents a serious problem with the static hashing technique of the previous section. Most databases grow larger over time. If we are to use static hashing for such a database, we have three classes of options:

1. Choose a hash function based on the current file size. This option will result in performance degradation as the database grows.
2. Choose a hash function based on the anticipated size of the file at some point in the future. Although performance degradation is avoided, a significant amount of space may be wasted initially.
3. Periodically reorganize the hash structure in response to file growth. Such a reorganization involves choosing a new hash function, recomputing the hash function on every record in the file, and generating new bucket assignments. This reorganization is a massive, time-consuming operation. Furthermore, it is necessary to forbid access to the file during reorganization.

Several **dynamic hashing** techniques allow the hash function to be modified dynamically to accommodate the growth or shrinkage of the database. In this section we describe one form of dynamic hashing, called **extendable hashing**. The bibliographical notes provide references to other forms of dynamic hashing.

24.5.2.1 Data Structure

Extendable hashing copes with changes in database size by splitting and coalescing buckets as the database grows and shrinks. As a result, space efficiency is retained. Moreover, since the reorganization is performed on only one bucket at a time, the resulting performance overhead is acceptably low.

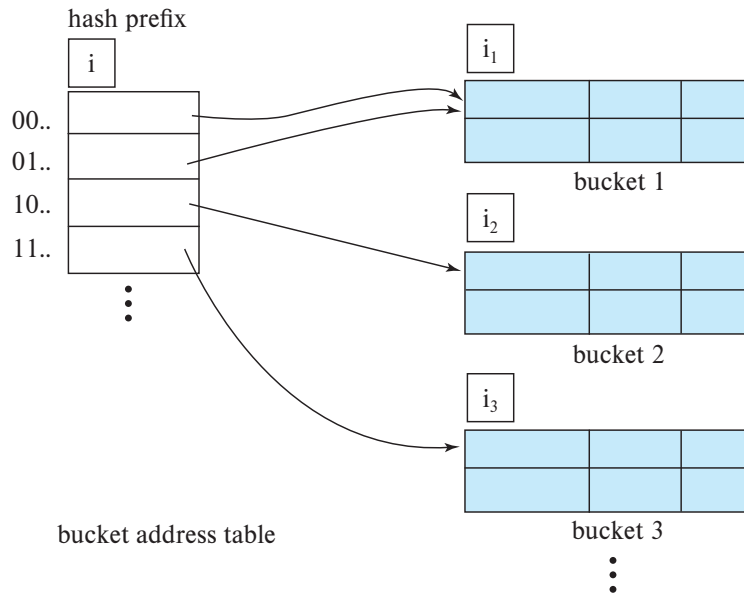


Figure 24.8 General extendable hash structure.

With extendable hashing, we choose a hash function h with the desirable properties of uniformity and randomness. However, this hash function generates values over a relatively large range—namely, b -bit binary integers. A typical value for b is 32.

We do not create a bucket for each hash value. Indeed, 2^{32} is over 4 billion, and that many buckets is unreasonable for all but the largest databases. Instead, we create buckets on demand, as records are inserted into the file. We do not use the entire b bits of the hash value initially. At any point, we use i bits, where $0 \leq i \leq b$. These i bits are used as an offset into an additional table of bucket addresses. The value of i grows and shrinks with the size of the database.

Figure 24.8 shows a general extendable hash structure. The i appearing above the bucket address table in the figure indicates that i bits of the hash value $h(K)$ are required to determine the correct bucket for K . This number will change as the file grows. Although i bits are required to find the correct entry in the bucket address table, several consecutive table entries may point to the same bucket. All such entries will have a common hash prefix, but the length of this prefix may be less than i . Therefore, we associate with each bucket an integer giving the length of the common hash prefix. In Figure 24.8 the integer associated with bucket j is shown as i_j . The number of bucket-address-table entries that point to bucket j is

$$2^{(i-i_j)}$$

24.5.2.2 Queries and Updates

We now see how to perform lookup, insertion, and deletion on an extendable hash structure.

To locate the bucket containing search-key value K_I , the system takes the first i high-order bits of $h(K_I)$, looks at the corresponding table entry for this bit string, and follows the bucket pointer in the table entry.

To insert a record with search-key value K_I , the system follows the same procedure for lookup as before, ending up in some bucket—say, j . If there is room in the bucket, the system inserts the record in the bucket. If, on the other hand, the bucket is full, it must split the bucket and redistribute the current records, plus the new one. To split the bucket, the system must first determine from the hash value whether it needs to increase the number of bits that it uses.

- If $i = i_j$, only one entry in the bucket address table points to bucket j . Therefore, the system needs to increase the size of the bucket address table so that it can include pointers to the two buckets that result from splitting bucket j . It does so by considering an additional bit of the hash value. It increments the value of i by 1, thus doubling the size of the bucket address table. It replaces each entry with two entries, both of which contain the same pointer as the original entry. Now two entries in the bucket address table point to bucket j . The system allocates a new bucket (bucket z) and sets the second entry to point to the new bucket. It sets i_j and i_z to i . Next, it rehashes each record in bucket j and, depending on the first i bits (remember the system has added 1 to i), either keeps it in bucket j or allocates it to the newly created bucket.

The system now reattempts the insertion of the new record. Usually, the attempt will succeed. However, if all the records in bucket j , as well as the new record, have the same hash-value prefix, it will be necessary to split a bucket again, since all the records in bucket j and the new record are assigned to the same bucket. If the hash function has been chosen carefully, it is unlikely that a single insertion will require that a bucket be split more than once, unless there are a large number of records with the same search key. If all the records in bucket j have the same search-key value, no amount of splitting will help. In such cases, overflow buckets are used to store the records, as in static hashing.

- If $i > i_j$, then more than one entry in the bucket address table points to bucket j . Thus, the system can split bucket j without increasing the size of the bucket address table. Observe that all the entries that point to bucket j correspond to hash prefixes that have the same value on the leftmost i_j bits. The system allocates a new bucket (bucket z), and sets i_j and i_z to the value that results from adding 1 to the original i_j value. Next, the system needs to adjust the entries in the bucket address table that previously pointed to bucket j . (Note that with the new value for i_j , not all the entries correspond to hash prefixes that have the same value on the leftmost i_j bits.) The system leaves the first half of the entries as they were (pointing to bucket

<i>dept_name</i>	$h(\text{dept_name})$
Biology	0010 1101 1111 1011 0010 1100 0011 0000
Comp. Sci.	1111 0001 0010 0100 1001 0011 0110 1101
Elec. Eng.	0100 0011 1010 1100 1100 0110 1101 1111
Finance	1010 0011 1010 0000 1100 0110 1001 1111
History	1100 0111 1110 1101 1011 1111 0011 1010
Music	0011 0101 1010 0110 1100 1001 1110 1011
Physics	1001 1000 0011 1111 1001 1100 0000 0001

Figure 24.9 Hash function for *dept_name*.

j), and sets all the remaining entries to point to the newly created bucket (bucket z). Next, as in the previous case, the system rehashes each record in bucket j , and allocates it either to bucket j or to the newly created bucket z .

The system then reattempts the insert. In the unlikely case that it again fails, it applies one of the two cases, $i = i_j$ or $i > i_j$, as appropriate.

Note that, in both cases, the system needs to recompute the hash function on only the records in bucket j .

To delete a record with search-key value K_j , the system follows the same procedure for lookup as before, ending up in some bucket—say, j . It removes both the search key from the bucket and the record from the file. The bucket, too, is removed if it becomes empty. Note that, at this point, several buckets can be coalesced, and the size of the bucket address table can be cut in half. The procedure for deciding on which buckets can be coalesced and how to coalesce buckets is left to you to do as an exercise. The conditions under which the bucket address table can be reduced in size are also left to you as an exercise. Unlike coalescing of buckets, changing the size of the bucket address table is a rather expensive operation if the table is large. Therefore it may be worthwhile to reduce the bucket-address-table size only if the number of buckets reduces greatly.

To illustrate the operation of insertion, we use the *instructor* file and assume that the search key is *dept_name* with the 32-bit hash values as appear in Figure 24.9. Assume that, initially, the file is empty, as in Figure 24.10. We insert the records one by one. To

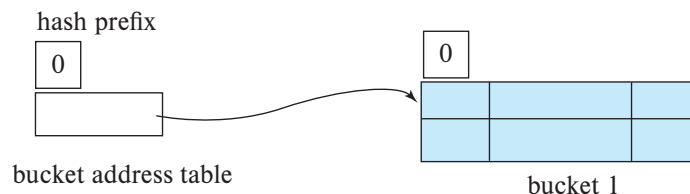


Figure 24.10 Initial extendable hash structure.

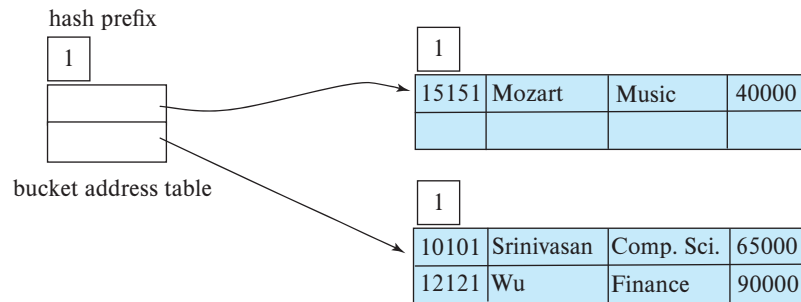


Figure 24.11 Hash structure after three insertions.

illustrate all the features of extendable hashing in a small structure, we shall make the unrealistic assumption that a bucket can hold only two records.

We insert the record (10101, Srinivasan, Comp. Sci., 65000). The bucket address table contains a pointer to the one bucket, and the system inserts the record. Next, we insert the record (12121, Wu, Finance, 90000). The system also places this record in the one bucket of our structure.

When we attempt to insert the next record (15151, Mozart, Music, 40000), we find that the bucket is full. Since $i = i_0$, we need to increase the number of bits that we use from the hash value. We now use 1 bit, allowing us $2^1 = 2$ buckets. This increase in the number of bits necessitates doubling the size of the bucket address table to two entries. The system splits the bucket, placing in the new bucket those records whose search key has a hash value beginning with 1, and leaving in the original bucket the other records. Figure 24.11 shows the state of our structure after the split.

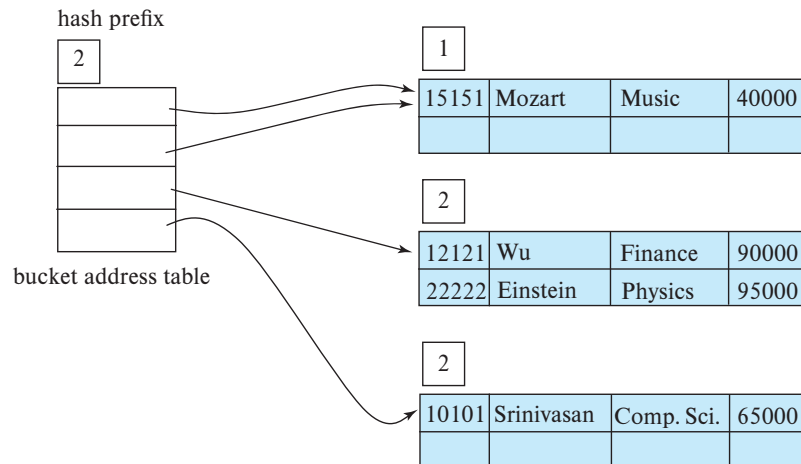


Figure 24.12 Hash structure after four insertions.

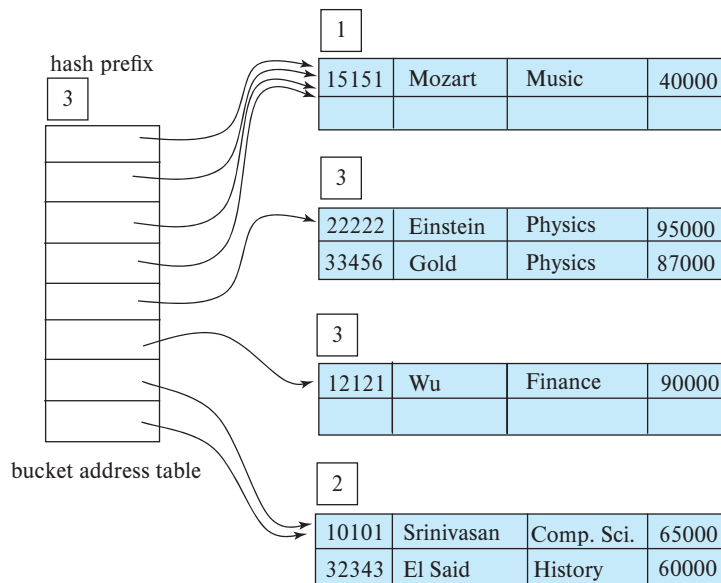


Figure 24.13 Hash structure after six insertions.

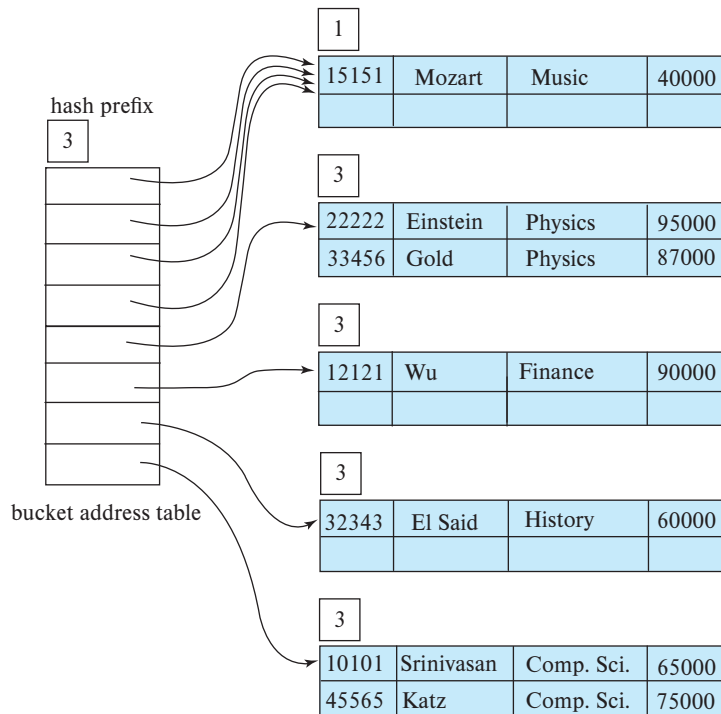


Figure 24.14 Hash structure after seven insertions.

Next, we insert (22222, Einstein, Physics, 95000). Since the first bit of $h(\text{Physics})$ is 1, we must insert this record into the bucket pointed to by the “1” entry in the bucket address table. Once again, we find the bucket full and $i = i_1$. We increase the number of bits that we use from the hash to 2. This increase in the number of bits necessitates doubling the size of the bucket address table to four entries, as in Figure 24.12. Since the bucket of Figure 24.11 for hash prefix 0 was not split, the two entries of the bucket address table of 00 and 01 both point to this bucket.

For each record in the bucket of Figure 24.11 for hash prefix 1 (the bucket being split), the system examines the first two bits of the hash value to determine which bucket of the new structure should hold it.

Next, we insert (32343, El Said, History, 60000), which goes in the same bucket as Comp. Sci. The following insertion of (33456, Gold, Physics, 87000) results in a bucket overflow, leading to an increase in the number of bits and a doubling of the size of the bucket address table (see Figure 24.13).

The insertion of (45565, Katz, Comp. Sci., 75000) leads to another bucket overflow; this overflow, however, can be handled without increasing the number of bits, since the bucket in question has two pointers pointing to it (see Figure 24.14).

Next, we insert the records of “Califieri”, “Singh”, and “Crick” without any bucket overflow. The insertion of the third Comp. Sci. record (83821, Brandt, Comp. Sci.,

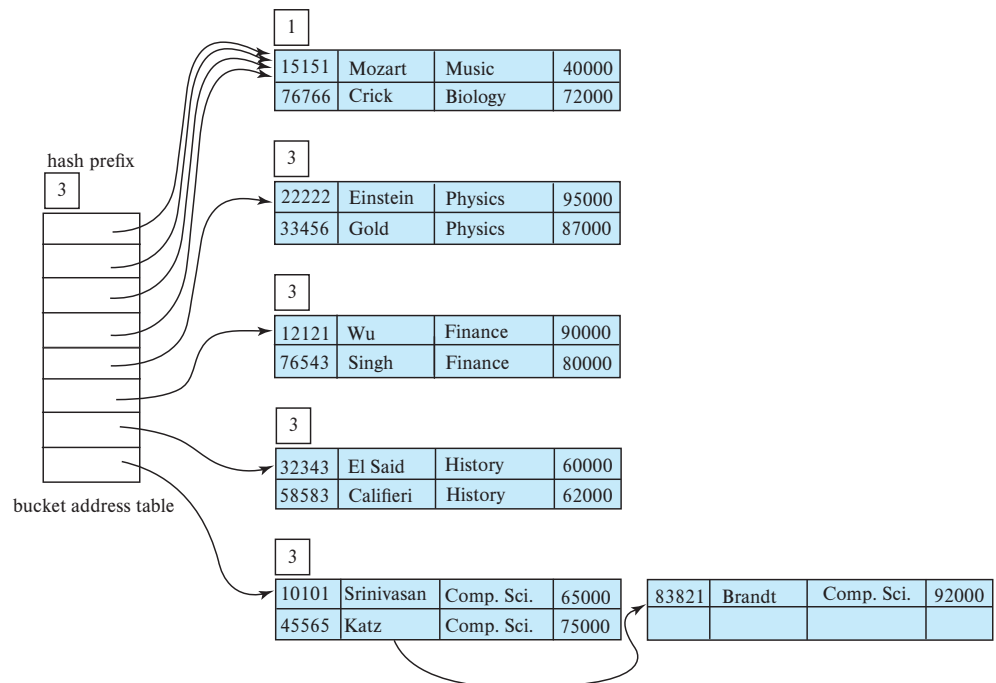


Figure 24.15 Hash structure after 11 insertions.

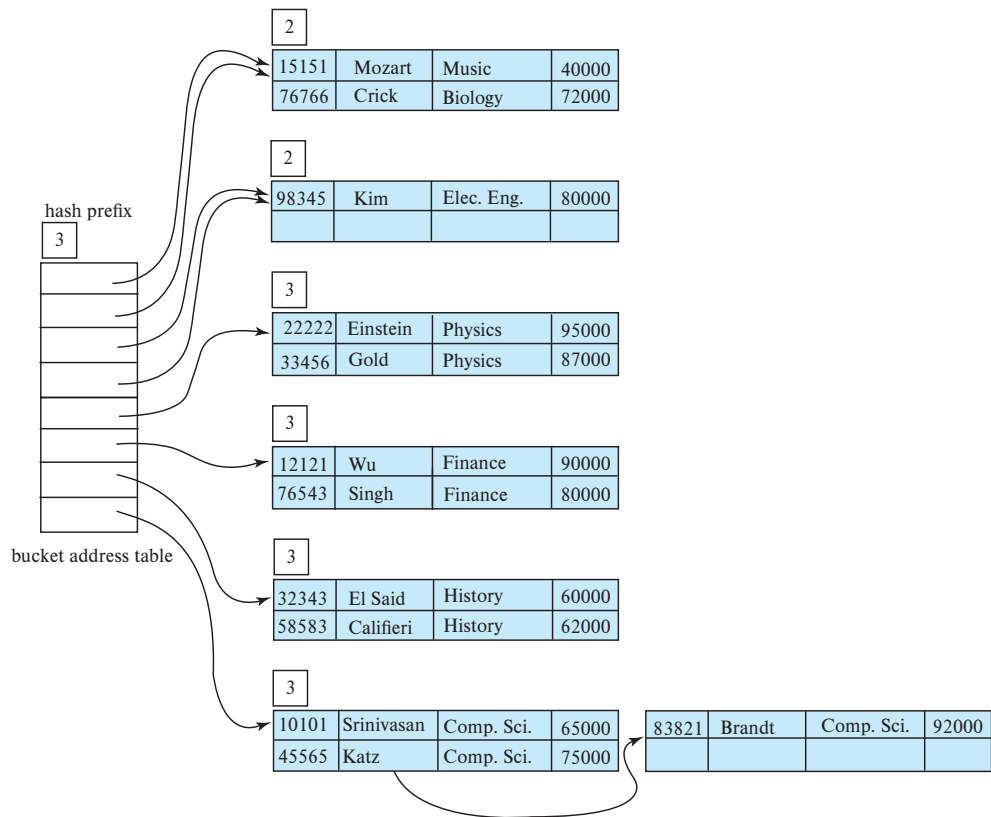


Figure 24.16 Extendable hash structure for the *instructor* file.

92000), however, leads to another overflow. This overflow cannot be handled by increasing the number of bits, since there are three records with exactly the same hash value. Hence the system uses an overflow bucket, as in Figure 24.15. We continue in this manner until we have inserted all the *instructor* records of Figure 14.1. The resulting structure appears in Figure 24.16.

24.5.2.3 Static Hashing versus Dynamic Hashing

We now examine the advantages and disadvantages of extendable hashing, compared with static hashing. The main advantage of extendable hashing is that performance does not degrade as the file grows. Furthermore, there is minimal space overhead. Although the bucket address table incurs additional overhead, it contains one pointer for each hash value for the current prefix length. This table is thus small. The main space saving of extendable hashing over other forms of hashing is that no buckets need to be reserved for future growth; rather, buckets can be allocated dynamically.

A disadvantage of extendable hashing is that lookup involves an additional level of indirection, since the system must access the bucket address table before accessing the